

Laboratorio di Calcolo

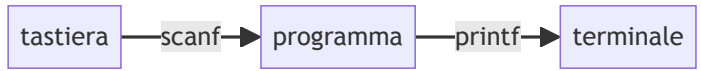
A.A. 2026/2027, Canale A-Dec

Giacomo Artoni, Lorenzo Caprini

Input, librerie e preprocessore

Perché leggere dei dati?

- Finora tutti i valori erano **scritti nel sorgente**: `double v0 = 3.0;`
- Per cambiare un dato bisognava **modificare il codice e ricompilare**.
- Un programma utile riceve i dati **dall'esterno**: dalla tastiera, da un file, da un altro programma.
- `printf` scrive sullo **standard output** (il terminale); `scanf` legge dallo **standard input** (la tastiera).
- Entrambe sono dichiarate in `<stdio.h>`: *standard input/output*.



Lo stesso eseguibile può ora risolvere **tanti problemi diversi**: basta cambiare i dati in ingresso, senza ricompilare.

La funzione scanf

`scanf` (*scan formatted*) legge dei valori e li **memorizza nelle variabili** indicate.

```c
#include <stdio.h>

int main() {
    double m, v;    // [kg], [m/s]

    printf("Massa [kg]: ");
    scanf("%lf", &m);
    printf("Velocita' [m/s]: ");
    scanf("%lf", &v);

    printf("K = %g J\n", 0.5 * m * v * v);
    return 0;
}
```

```
$ ./energia
Massa [kg]: 2
Velocita' [m/s]: 3
K = 9 J
```

- La stringa di formato dice **che tipo** di valore leggere: `%lf` per un `double`.
- `&m` dice **dove** metterlo: nella variabile `m`.
- Il programma si **ferma** e aspetta che l'utente scriva un valore e prema **Invio**.
- Prima di ogni `scanf`, una `printf` spiega all'utente **cosa** inserire (e in che unità!).

Specificatori di scanf

Gli specificatori somigliano a quelli di `printf`, ma per i reali le regole sono **diverse**.

Tipo	`printf`	`scanf`
`int`	`%d`	`%d`
`long`	`%ld`	`%ld`
`unsigned int`	`%u`	`%u`
`float`	`%f`	`%f`
`double`	`%f`	`%lf`
`char`	`%c`	`%c`

- In `printf` un `float` viene convertito in `double`, quindi `%f` va bene per entrambi (lezione 12).
- `scanf` invece deve **scrivere** nella variabile: deve sapere se occupa **4** byte (`float`) o **8** byte (`double`).
- La `l` di `%lf` sta per *long float*, cioè `double`.
- `%e` e `%g` in `scanf` equivalgono a `%f`: tutti accettano `2.5`, `2.5e-3`, `7`.

Regola pratica: `double` **si legge con** `%lf`, sempre.

L'operatore &

- `printf("%g", m)` riceve il **valore** di `m`: le basta leggerlo.
- `scanf` deve **modificare** `m`: non le serve il valore, ma la **posizione in memoria** della variabile.
- `&m` è l'**indirizzo** di `m`: il numero della "cella" di memoria in cui è conservata (lezione 5).
- Con l'indirizzo, `scanf` sa **dove** scrivere il valore letto.
- Gli indirizzi (i *puntatori*) saranno l'argomento di una lezione successiva: per ora ricordate solo che in `scanf` ci vuole la `&`.

```
// memoria (schematica)

indirizzo    contenuto    nome
0×7ffc10     2.0          m
0×7ffc18     3.0          v

m   → 2.0    (il valore)
&m → 0×7ffc10   (l'indirizzo)
```

Gli indirizzi mostrati sono di fantasia: cambiano da macchina a macchina e da un'esecuzione all'altra.

Leggere più valori

```c
#include <stdio.h>

int main() {
    int n = -1;
    double x = -1.0;

    int letti = scanf("%d %lf", &n, &x);

    printf("letti = %d\n", letti);
    printf("n = %d, x = %g\n", n, x);
    return 0;
}
```

- Una sola `scanf` può leggere **più valori**, nell'ordine dei segnaposto.
- Spazi, tabulazioni e "a capo" tra i numeri vengono **saltati**.
- `scanf` **restituisce** il numero di valori letti con successo.

Input	`letti`	`n`	`x`
`5 2.5`	`2`	`5`	`2.5`
`5` ↵ ↵ `2.5`	`2`	`5`	`2.5`
`5 ciao`	`1`	`5`	`-1`
`ciao`	`0`	`-1`	`-1`

Se la lettura fallisce, la variabile **resta com'era**: se non era inizializzata, contiene un valore a caso! Il valore restituito permette di **accorgersene**: impareremo a controllarlo nella prossima lezione.

Errori tipici con scanf

1. %f per un double

```
double t = 0.0;      // [s]
scanf("%f", &t);
printf("t = %g s\n", t);
```

```
warning: format ‘%f’ expects argument of type
  ‘float *’, but argument 2 has type ‘double *’
  [-Wformat=]
$ ./sbagliato
Tempo [s]: 2.5
t = 5.31535e-315 s
```

`scanf` ha scritto un `float` (4 byte) dentro un `double` (8 byte): i bit sono interpretati male.

2. & dimenticata

```
int n = 0;
scanf("%d", n);
```

```
warning: format ‘%d’ expects argument of type
  ‘int *’, but argument 2 has type ‘int’
  [-Wformat=]
$ ./noamp
Numero di misure: 10
Segmentation fault (core dumped)
```

`scanf` usa il **valore** `0` come se fosse un indirizzo e prova a scriverci: il sistema operativo blocca il programma.

Entrambi gli errori compilano! Solo `-Wall` li segnala.

Il preprocessore e #include

Le righe che iniziano con `#` sono **direttive al preprocessore**: vengono eseguite **prima** della compilazione vera e propria (lezione 8).

- `#include <file.h>` copia nel sorgente il contenuto di un **file di intestazione** (*header*).
- Un header contiene le **dichiarazioni** delle funzioni: nome, tipo degli argomenti, tipo restituito.
- Così il compilatore può controllare che le funzioni siano usate correttamente.
- Il **codice** delle funzioni non è nell'header: è in una **libreria** già compilata, che aggiunge il linker.

Direttiva	Dove cerca il file
`#include <stdio.h>`	Cartelle di **sistema** (es. `/usr/include`)
`#include "miofile.h"`	Prima nella **cartella del sorgente**, poi in quelle di sistema

```
1   $ gcc -E hello.c | less
```

- `gcc -E` mostra il risultato del preprocessore: centinaia di righe prese da `stdio.h`, prima del nostro `main`.
- Gli header scritti da noi (`"..."`) serviranno quando divideremo un programma in più file.

La libreria matematica math.h

Funzione	Significato
`sqrt(x)`	$\sqrt{x}$
`pow(x, y)`	x^y
`exp(x)` , `log(x)` , `log10(x)`	e^x, $\ln x$, $\log_{10} x$
`sin(x)` , `cos(x)` , `tan(x)`	funzioni trigonometriche (**radianti**!)
`asin(x)` , `acos(x)` , `atan(x)`	funzioni inverse
`atan2(y, x)`	angolo del punto (x, y), in $(-\pi, \pi]$
`fabs(x)`	$\lvert x \rvert$ per i `double`
`floor(x)` , `ceil(x)`	arrotonda per difetto / per eccesso
`M_PI` , `M_E`	le costanti π ed e

```c
// valori stampati con %f
sqrt(2.0)        //  1.414214
pow(2.0, 0.5)    //  1.414214
exp(1.0)         //  2.718282
log(M_E)         //  1.000000
sin(M_PI / 6)    //  0.500000
sin(30.0)        // -0.988032
fabs(-3.5)       //  3.500000
```

- Argomenti e risultati sono `double` .
- `sin(30.0)` è il seno di 30 **radianti**: i gradi vanno convertiti, $\theta_{\text{rad}} = \theta \cdot \pi / 180$.
- `M_PI` non fa parte dello standard C, ma su Linux c'è sempre.

Esempio: gittata di un proiettile

```c
#include <stdio.h>
#include <math.h>

int main() {
    const double g = 9.81;      // [m/s^2]
    double v0, theta_gradi;     // [m/s], [gradi]

    printf("v0 [m/s] e angolo [gradi]: ");
    scanf("%lf %lf", &v0, &theta_gradi);

    double theta = theta_gradi * M_PI / 180.0;  // [rad]
    double gittata = v0 * v0 * sin(2 * theta) / g;
    double h_max = pow(v0 * sin(theta), 2) / (2 * g);

    printf("Gittata:         %.2f m\n", gittata);
    printf("Altezza massima: %.2f m\n", h_max);
    return 0;
}
```

```
$ gcc -Wall -Wextra -o gittata gittata.c -lm
$ ./gittata
v0 [m/s] e angolo [gradi]: 10 45
Gittata:         10.19 m
Altezza massima: 2.55 m
```

- $R = \dfrac{v_0^2 \sin 2\theta}{g}, \quad h_{\max} = \dfrac{(v_0 \sin\theta)^2}{2g}$
- **Test:** con $\theta = 45^\circ$ e $v_0 = 10$ m/s, $R = 100/9.81 = 10.19$ m ✓
- **Test:** con $\theta = 90^\circ$ la gittata deve essere zero: il programma stampa `0.00` ✓
- Notate `-lm` in fondo al comando di compilazione…

Collegare la libreria: -lm

```
$ gcc -Wall -Wextra -o gittata gittata.c
/usr/bin/ld: /tmp/ccXdP3Qa.o: in function `main':
gittata.c:(.text+0xa5): undefined reference to `sin'
collect2: error: ld returned 1 exit status
```

- Il **compilatore** è contento: grazie a `math.h` conosce `sin`.
- Il **linker** non trova il **codice** di `sin`: è un *errore di linking* (lezione 7).
- `printf` e `scanf` stanno nella libreria C standard, collegata automaticamente; le funzioni matematiche no.

- `-lm` chiede al linker di usare la libreria **m** (matematica), il file `libm.so`.
- L'opzione va messa **dopo** il file sorgente:

```
$ gcc -Wall -Wextra -o gittata gittata.c -lm
```

`#include <math.h>` serve al **compilatore** (le dichiarazioni), `-lm` serve al **linker** (il codice). Servono **entrambi**.

Costanti simboliche: #define

```c
#define G     9.81      // [m/s^2]
#define N_MAX 1000

double peso = m * G;
```

- `#define NOME testo` : il preprocessore **sostituisce** ogni `NOME` con `testo` , prima della compilazione.
- `NOME` si chiama **macro**: per convenzione si scrive in MAIUSCOLO.
- Niente `=` e niente `;` : non è un'istruzione C!
- Dopo il preprocessore, il compilatore vede solo `m * 9.81` .

	`#define`	`const` (lezione 10)
Chi la gestisce	preprocessore	compilatore
Ha un tipo?	no, è testo	sì
Occupa memoria?	no	sì, è una variabile
Visibilità	dal `#define` in poi	il blocco in cui è dichiarata
Ridefinibile con `-D`	sì	no

Entrambe vanno bene per le costanti: `const` è più sicura (il compilatore controlla il tipo), `#define` è più flessibile.

La trappola delle macro

Due resistenze in serie: $R = R_1 + R_2 = 320\ \Omega$, con $I = 0.01\ \mathrm{A}$ ci aspettiamo $V = 3.2\ V$.

```c
#define R1 100.0              // [ohm]
#define R2 220.0              // [ohm]
#define R_SERIE R1 + R2       // SBAGLIATO!

double i = 0.01;              // [A]
double v = i * R_SERIE;       // [V]
printf("V = %g V\n", v);
```

```
V = 221 V
```

```
$ gcc -E serie.c | tail -4
    double i = 0.01;
    double v = i * 100.0 + 220.0;
```

- La sostituzione è **puramente testuale**: la precedenza fa il resto (lezione 11).
- Rimedio: **sempre le parentesi** attorno al testo della macro:

```c
#define R_SERIE (R1 + R2)
```

Anche un `;` di troppo (`#define G 9.81;`) viene copiato ovunque, e produce errori di sintassi in punti misteriosi.

Compilazione condizionata

Il preprocessore può **includere o escludere** parti del codice.

```c
#include <stdio.h>
#include <math.h>

#ifndef G
#define G 9.81      // [m/s^2] valore sulla Terra
#endif

int main() {
    double l;       // lunghezza [m]

    printf("Lunghezza del pendolo [m]: ");
    scanf("%lf", &l);

#ifdef DEBUG
    printf("DEBUG: l = %g m, G = %g m/s^2\n", l, G);
#endif

    printf("Periodo: %.3f s\n", 2 * M_PI * sqrt(l / G));
    return 0;
}
```

- `#ifdef NOME` … `#endif`: codice compilato **solo se** `NOME` è definita; `#ifndef`: **solo se non** lo è (qui `G` vale 9.81 se nessuno l'ha definita).
- Le stampe di `DEBUG` aiutano a cercare errori e nella versione finale **spariscono** senza cancellarle.
- Per **attivarle**: `#define DEBUG` in cima al file (basta il nome, senza valore), oppure dalla riga di comando (prossima slide).

Non è un `if` del C: la scelta avviene **prima** della compilazione, non durante l'esecuzione.

Macro dalla riga di comando: -D

L'opzione `-DNOME` definisce una macro **al momento della compilazione**, senza toccare il sorgente.
Con `-DNOME=valore` le si dà anche un valore.

```
$ gcc -Wall -Wextra -o pendolo pendolo.c -lm
$ ./pendolo
Lunghezza del pendolo [m]: 1
Periodo: 2.006 s
```

```
$ gcc -Wall -Wextra -DG=1.62 -o pendolo pendolo.c -lm
$ ./pendolo
Lunghezza del pendolo [m]: 1
Periodo: 4.937 s
```

```
$ gcc -Wall -Wextra -DDEBUG -o pendolo pendolo.c -lm
$ ./pendolo
Lunghezza del pendolo [m]: 1
DEBUG: l = 1 m, G = 9.81 m/s^2
Periodo: 2.006 s
```

- **Test:** $T = 2\pi\sqrt{1/9.81} = 2.006$ s ✓
- Con `-DG=1.62` lo stesso sorgente calcola il periodo **sulla Luna**.
- Con `-DDEBUG` compaiono le stampe di controllo.
- Le opzioni si possono combinare: `-DDEBUG -DG=1.62`

Siete stati attenti?

Cosa c'è che non va nel codice qui sotto?

```c
#include <stdio.h>
#define N 10;
#define DIST 3 + 1

int main() {
    double r;   // [m]
    int k;
    scanf("%f", &r);          // (a)
    scanf("%d", k);           // (b)
    double y = sqrt(r);       // (c)
    int a = 2 * DIST;         // (d)
    int b = N + 1;            // (e)
    return 0;
}
```

Riga	Problema
`(a)`	`r` è `double`: serve `%lf`
`(b)`	Manca la `&`: crash a runtime
`(c)`	Manca `#include <math.h>` (e `-lm`)
`(d)`	Diventa `2 * 3 + 1`, cioè `7` e non `8`: servono le parentesi
`(e)`	Diventa `10; + 1;`: compila (con un warning), ma `b` vale `10`